PRACTICAL: 10

Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Objective:

To develop secure login and logout functionality in PHP using sessions. The practical covers verifying passwords with password_verify(), regenerating the session ID after login, restricting dashboard access to logged-in users, showing role-based (admin/student) dashboards, and ending idle sessions with a session timeout.

Theory:

Authentication confirms who a user is, and a **session** remembers that identity across requests. PHP stores session data on the server and gives the browser only a session ID cookie, so no password or role is kept on the client.

- **Session start and destroy:** session_start() begins the session. On logout (and on timeout) $_SESSION is emptied and session_destroy() is called, so the old session can no longer be used.
- **Password verification:** Passwords are stored with password_hash() in a VARCHAR(255) column. At login, password_verify($password, $hash) compares the typed password with the stored hash; the plain password is never stored or compared directly.
- **Session fixation protection:** session_regenerate_id(true) is called right after a successful login, so an ID known before login becomes useless.
- **Prepared statements:** The username lookup uses mysqli_prepare() and bind_param(), which prevents SQL injection in the login form.
- **Restricted dashboard:** The dashboard is displayed only when $_SESSION['user_id'] is set. Otherwise the login form is shown.
- **Role-based access:** The role (admin or student) is read from the users table at login, stored in $_SESSION['role'], and used to show the Admin or Student dashboard.
- **Session timeout:** The time of the last request is kept in $_SESSION['last_activity']. If more than 300 seconds (5 minutes) pass without activity, the session is destroyed and the user is sent to login.php?timeout=1, which shows "Session expired. Please login again."
- **Output escaping:** Error messages are printed with htmlspecialchars() to avoid XSS.

Note: the optional extensions (last-login timestamp and remember-me token) are not part of this implementation.

Code:

Login.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<link rel="stylesheet" href="style.css">
<title>StudentHub - Login</title>
</head>


<body>


<header>
<h1>StudentHub</h1>
</header>


<nav aria-label="Main Navigation">
<a href="index.html">Home</a> |
<a href="Register.html">Register</a> |
<a href="Login.html">Login</a>
</nav>


<main>
<section>
<h2>Student Login</h2>


<form>


<label for="email">Email:</label>
<input type="email" id="email" name="email" required>
<br><br>


<label for="password">Password:</label>
<input type="password" id="password" name="password" required>

<br><br>


<button type="submit">Login</button>


</form>
</section>
</main>




</body>
</html>
```

Secure-login.html

secure-login.php

```php
<?php
require_once __DIR__ . '/config.php';
if (!empty($_SESSION['user_id'])) {
    header('Location: ' . (($_SESSION['role'] ?? '') === 'admin' ? 'admin-dashboard.php' : 'student-dashboard.php'));
    exit;
}
$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = strtolower(trim($_POST['email'] ?? ''));
    $password = $_POST['password'] ?? '';
    if (filter_var($email, FILTER_VALIDATE_EMAIL) && $password !== '') {
        $stmt = $conn->prepare('SELECT id, name, username, email, password, course, role FROM users WHERE email = ? LIMIT 1');
        $stmt->bind_param('s', $email);
        $stmt->execute();
        $user = $stmt->get_result()->fetch_assoc();
        if ($user && password_verify($password, $user['password'])) {
            session_regenerate_id(true);
            $_SESSION['user_id'] = (int)$user['id'];
            $_SESSION['name'] = $user['name'];
            $_SESSION['username'] = $user['username'];
            $_SESSION['email'] = $user['email'];
            $_SESSION['course'] = $user['course'];
            $_SESSION['role'] = $user['role'];
            $_SESSION['last_activity'] = time();
            $update = $conn->prepare('UPDATE users SET last_login = NOW() WHERE id = ?');
            $update->bind_param('i', $_SESSION['user_id']);
            $update->execute();
            header('Location: ' . ($user['role'] === 'admin' ? 'admin-dashboard.php' : 'student-dashboard.php'));
            exit;
        }
    }
    $error = 'Invalid email or password.';
}
?>


<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <link rel="stylesheet" href="style.css">

    <title>StudentHub - Login</title>
</head>

```

```
    <link rel="stylesheet" href="style.css">

    <title>StudentHub - Login</title>
</head>

<body>

    <header>
        <h1>StudentHub</h1>
    </header>

    <nav aria-label="Main Navigation">
        <a href="index.html">Home</a> |
        <a href="secure-register.php">Register</a> |
        <a href="secure-login.php">Login</a>
    </nav>

    <main>
        <section>

            <h2>Student Login</h2>

            <?php if ($error !== ''): ?>
                <p role="alert">
                    <?= h($error) ?>
                </p>
            <?php endif; ?>

            <?php if (isset($_GET['timeout'])): ?>
                <p role="alert">
                    Your session expired after five minutes of inactivity.
                    Please log in again.
                </p>
            <?php endif; ?>

            <?php if (isset($_GET['required'])): ?>
                <p role="alert">
                    Please log in to access that page.
                </p>
            <?php endif; ?>

            <form method="post" action="secure-login.php">

                <label for="email">Email:</label>
                <input
                    type="email"
                    id="email"
```

```
                required
            >

            <br><br>

            <label for="password">Password:</label>
            <input
                type="password"
                id="password"
                name="password"
                required
            >

            <br><br>

            <button type="submit">Login</button>

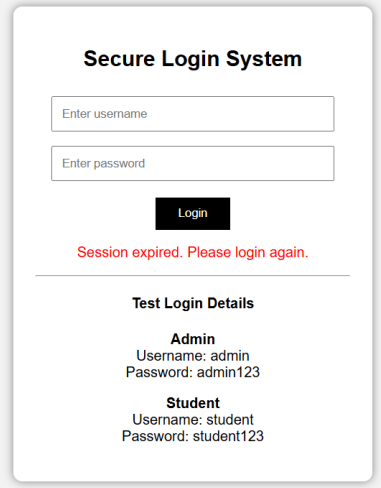
        </form>

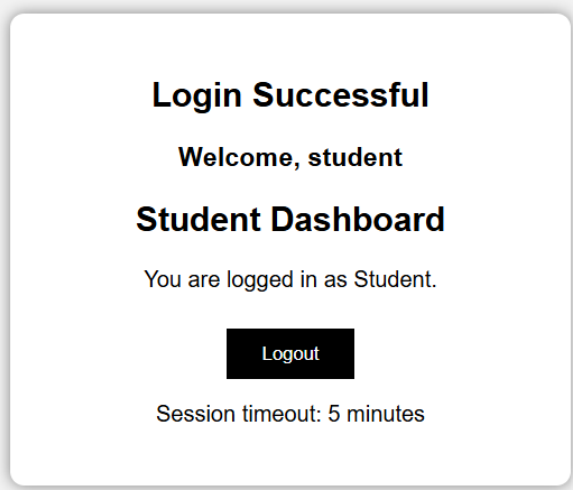
        <p>
            <a href="secure-register.php">
                Create a student account
            </a>
        </p>

    </section>
</main>

</body>
</html>
```

Output:





Learning Outcome:

Students implemented secure login and logout with PHP sessions. They learned to verify hashed passwords with password_verify(), regenerate the session ID after login, destroy sessions properly, restrict the dashboard to authenticated users, show different dashboards for admin and student roles, and end idle sessions using a 5-minute timeout.

References:

- PHP Manual – Sessions: https://www.php.net/manual/en/book.session.php
- PHP Manual – password_verify(): https://www.php.net/manual/en/function.password-verify.php
- PHP Manual – session_regenerate_id(): https://www.php.net/manual/en/function.session-regenerate-id.php